Using the InterSystems SQL Gateway

Version 2026.1
2026-04-20

InterSystems Corporation   One Congress Street   Boston MA 02114   www.intersystems.com

Using the InterSystems SQL Gateway
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1

SQL Gateway Overview

See the Table of Contents for a detailed listing of the subjects covered in this document.

The InterSystems SQL Gateway provides access from InterSystems IRIS® data platform to external databases. You can use various wizards to create links to tables, views, or stored procedures in external sources, allowing you to access the data in the same way you access any InterSystems IRIS object:

- Access data stored in third-party relational databases within InterSystems IRIS applications using objects and/or SQL queries.
- Store persistent InterSystems IRIS objects in external relational databases.
- Create class methods that perform the same actions as corresponding external stored procedures.
- Connect through either the InterSystems JDBC driver or the InterSystems ODBC driver.

Related Documents

The following documents contain related material:

- *Using InterSystems SQL* — describes how to use InterSystems SQL, which provides standard relational access to data stored in an InterSystems IRIS database.
- *Using Java with InterSystems Software* — provides an overview of all InterSystems Java technologies enabled by the InterSystems JDBC driver, and describes how to use the driver to access data sources via SQL.
- *Using the InterSystems ODBC Driver* — describes how to connect to InterSystems IRIS from an external application via InterSystems ODBC, and how to access external ODBC data sources from InterSystems IRIS.

2
Conn SQL Gateway

3

Using Linked Tables and Linked Procedures

SQL Gateway connections are used to access remote tables and procedures created by the Link Table Wizard and Link Procedure Wizard, either programmatically or as part of an interoperability function.

- The Link Table Wizard: Linking to a Table or View — describes the procedure for linking to tables or views in external sources so that you can access the data in the same way you access any InterSystems IRIS object.
- The Link Procedure Wizard: Linking to a Stored Procedure — describes the procedure for linking to stored procedures in external sources.

3.1 The Link Table Wizard: Linking to a Table or View

The Management Portal provides a wizard that you can use to link to an external table in an ODBC- or JDBC-compliant database. When you have linked to an external table, you can:

- Access data stored in third-party relational databases within InterSystems IRIS applications using objects and/or SQL queries.
- Store persistent InterSystems IRIS objects in external relational databases.

For example, suppose you have an `Employee` table stored within an external relational database. You can use this table within InterSystems IRIS as an object by creating an `Employee` class that communicates (by executing SQL queries via JDBC or ODBC) with the external database.

From the perspective of an InterSystems IRIS application, the `Employee` class behaves in much the same way as any other persistent class: You can open instances, modify, and save them. If you issue SQL queries against the `Employee` class, they are automatically dispatched to the external database.

The use of the InterSystems SQL Gateway is independent of application logic; an application can be modified to switch between external databases and the built-in InterSystems IRIS database with minimal effort and no change to application logic.

Any class that uses the InterSystems SQL Gateway to provide object persistence is identical in usage to classes that using native persistence and can make full use of InterSystems IRIS features including Java, SQL, and Web access.

Note: The information captured in an SQL Gateway definition can also be used to create foreign tables, which is a pure SQL approach to projecting data from remote databases or files to InterSystems IRIS SQL. This page describes only how to use the Link Table Wizard to access external tables; to read more about using foreign tables, see Foreign Tables.

3.1.1 Using the Link Table Wizard

When you link to an external table or view, you create a persistent InterSystems IRIS class that is linked to that table or view. The new class stores and retrieves data from the external source using the SQL Gateway. You can specify information about both the InterSystems IRIS class and the corresponding SQL table in InterSystems IRIS.

Note: This wizard generates ObjectScript code with class names and class member names that you control. When you use this wizard, be sure to follow the rules for ObjectScript identifiers, including length limits (see the section on Naming Conventions in *Defining and Using Classes*).

- If you have not yet created a connection to the external database, do so before you begin (see "Creating SQL Gateway Connections").
- From the Management Portal select **System Explorer**, then **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces.

  At the top of the page, click the `Wizards` drop-down list, and select **Link Table**.
- On the first page of the wizard, select one or more table or views, as follows:
  - `Select a destination namespace` — Select the InterSystems IRIS namespace to which the data will be copied.
  - `Schema Filter` — Specify a schema (class package) name that contains the table or view. You can specify a name with wildcards to return multiple schemas, or % to return all schemas. For example, C% will return all schemas in the namespace beginning with the letter C. Use of this filter is recommended, as it will shorten the return list of schemas to select from, and thus improve loading speed. You can select multiple items. In this case, when you click `Next`, the next screen prompts you for a package name. Specify the name of the package to contain the classes and then click `Finish`.
  - `Table Filter` — Specify the table or view to link to. You can specify a name with wildcards to return multiple tables and/or views, or % to return all tables/views.
  - `Table type` — Select `TABLE`, `VIEW`, `SYSTEM TABLE`, or `ALL`. The default is `TABLE`.
  - `Select a SQL Gateway connection` — Select the SQL Gateway connection to use.
- Click `Next`.
- On the second page, specify which fields should be available as object properties in InterSystems IRIS. Make changes as follows:
  - Highlight one or more fields and click the single arrow to move it or them from one list to another; click the double arrow to move all fields (selected or not) from one list to another.
  - In the selected list, use the up and down arrows to modify the order of the fields in the table that InterSystems IRIS projects for the given class. This does not affect the order of the properties in the class definition.
- Click `Next`.
- On the third page, specify information about the properties in the generated class. For each property, you can specify all the available options:

  - `Read only` — Select this check box to make the property read-only. This controls the `ReadOnly` keyword for the property.

    Tip: Use the `select_all` check box to select or clear all the check boxes in this column.

  - `New Property Name` — Specifies the name of the object property that will contain the data from this field.
  - `New Column Name (SQL Field Name)` — Specifies the SQL field name to use for this property. This controls the `SqlFieldName` keyword for the property.

- Click `Next`.
- On the last page, specify the following:
  - `Primary Key` — Select the primary key for the new InterSystems IRIS table from the list provided. In addition to the default key provided, you can click the "Browse" button to select one or more columns. You may select multiple columns; multiple columns are returned as a composite key separated by commas. You must specify a primary key.
  - `New class name` — Specify the name of the InterSystems IRIS class to create, including the package. The default package name is `nullschema`.
  - `New table name` — Specify the name of the SQL table to create in InterSystems IRIS. This controls the `SqlTableName` keyword for the class.
- Click `Finish`. The wizard displays the Background Jobs page with a link to the background tasks page.
- Click `Close`. Or click the given link to view the background tasks page. In either case, the wizard starts a background task to do the work.

The wizard stores a new class definition in the InterSystems IRIS database and compiles it. If data is present, it should be immediately visible in the external database (you can check by issuing SQL queries against the newly created InterSystems IRIS class/table). You can now use the new class as you would any other persistent class within InterSystems IRIS.

Note: **Closing the Link Table Connection**

By design, the code generated by the Link Table Wizard does not close the connections that it opens. This avoids problems such as conflicts between SQL statements that share the same connection. See "Managing SQL Gateway Connections" for more information.

3.1.2 Limitations When Using a Linked Table

As always, it is important to be aware of the particular limitations (syntactical or otherwise) and requirements of the database to which you are connecting. The following are a few examples:

- JDBC connections, unlike ODBC, do not support heterogeneous INSERT...SELECT statements (involving both a local and a linked table, or two linked tables).
- ODBC connections, unlike JDBC, do not fill the %ROWID property.
- Informix: You cannot create a view inside of InterSystems SQL that is based on a linked Informix table, because the generated SQL is not valid in Informix.
- Sybase: As part of query processing, InterSystems SQL can transform the expression of an outer join into an equivalent canonicalized form. The SQL92-standard CROSS JOIN syntax may be required to reconstruct this form as SQL in order to access a linked table. Because Sybase does not support SQL92-standard CROSS JOIN, some queries using outer joins on linked Sybase tables will fail to execute.
- Oracle: the CAST must be performed around any COUNT aggregates used in Oracle sourced tables.

- MySQL: when accessing a stored procedure with INOUT and OUT, bound parameters are not updated correctly. Instead, the value is returned in the result set.

Before you try to use a linked table, you might want to examine the cached query that is generated for it, to ensure that the syntax is valid for the database you are using. To see the cached query for a given linked table:

- In the Management Portal, go to **System Explorer**, **SQL**.
- Click the namespace you are interested in.
- Select the Schema from the pull-down list.
- Click `Cached Queries` for the package that contains the table. The system displays a table of the cached queries for this package. The `Query` column displays the full query.
- Optionally click the link for the query to see more details.

3.1.3 Restrictions on SQL Gateway Queries

When you use the InterSystems SQL Gateway, note the following restrictions:

- Queries that join data from heterogeneous data sources are only supported for ODBC connections. When using JDBC, all tables listed in the `FROM` clause of an SQL query must come from the same data source. .
- SQL queries targeted at external databases cannot use the following InterSystems SQL extensions:
  - The "`->`" operator.
  - The %EXACT function, or the %SYSTEM.Util **Collation()** method with the collation flag set to EXACT.
  - The inclusion of other columns within a count (*) query.
  - InterSystems IRIS-specific operators that have `%` as the first character of their name.

3.2 The Link Procedure Wizard: Linking to a Stored Procedure

The Management Portal provides a wizard that you can use to link to a stored procedure defined in an external ODBC- or JDBC-compliant database. When you link to the procedure, the system generates a method and a class to contain the method. When you link to an stored procedure, you create a class method that does the same action that the stored procedure does. This method is marked with the `SqlProc` keyword. The class method is generated within a new class, and you can specify information such as the class and package name. This method cannot accept a variable number of arguments. Default parameters are permitted, but the signature of the stored procedure is fixed.

Note: **Closing the Link Procedure Connection**

By design, the code generated by the Link Procedure Wizard does not close the connections that it opens. This avoids problems such as conflicts between SQL statements that share the same connection. See “Managing SQL Gateway Connections” for more information.

- If you have not yet created a connection to the external database, do so before you begin (see “Creating SQL Gateway Connections”).

- From the Management Portal select **System Explorer**, then **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces.

  At the top of the page, click the `Wizards` drop-down list, and select **Link Procedure**.

- On the first page of the wizard, select one or more procedures, as follows:
  - `Select a destination namespace` — Select the InterSystems IRIS namespace to which the data will be copied.
  - `Schema Filter` — Specify a schema (class package) name that contains the procedure. You can specify a name with wildcards to return multiple schemas, or % to return all schemas. For example, C% will return all schemas in the namespace beginning with the letter C. Use of this filter is recommended, as it will shorten the return list of schemas to select from, and thus improve loading speed.
  - `Procedure Filter` — Specify a procedure to link to. You can specify a name with wildcards to return multiple procedures, or % to return all procedures. You can select multiple procedures. In this case, when you click `Next`, the next screen prompts you for a package name. Specify the name of the package to contain the classes and then click `Finish`.
  - `Select a SQL Gateway connection` — Select the SQL Gateway connection to use.
- Click `Next`.
- On the second page, specify details about the class to generate in InterSystems IRIS:
  - `New package name` — Specify the name of the package to contain the class or classes.
  - `New class name` — Specify the name of the class to generate.
  - `New procedure name` — Specify the name of the procedure; specifically this controls the `SqlName` keyword of the method.
  - `New method name` — Specify the name of the method to generate.
  - `Description method name` — Optionally provide a description of the method; this is used as a comment for the class definition, to be displayed in the class reference.
- Click `Finish`. The wizard displays the Background Jobs page with a link to the background tasks page.
- Click `Close`. Or click the given link to view the background tasks page. In either case, the wizard starts a background task to do the work.

The wizard stores a new class definition within the InterSystems IRIS database and compiles it.

Note: This wizard generates ObjectScript code with class names and class member names that you control. When you use this wizard, be sure to follow the rules for ObjectScript identifiers, including length limits (see the section on Naming Conventions in *Defining and Using Classes*).

4

Using the Data Migration Wizard

The Management Portal provides a wizard that you can use to migrate data from an external table or view.

When you migrate data from a table or view in an external source, the system generates a persistent class to store data of that table or view and then copies the data. This wizard assumes that the class should have the same name as the table or view from which it comes; similarly, the property names are the same as in the table or view. After the class has been generated, it does not have any connection to external data source.

- If you have not yet created an SQL Gateway connection to the external database, do so before you begin (see "Creating SQL Gateway Connections").
- From the Management Portal select **System Explorer**, then **SQL**. If necessary, change namespaces by clicking the current namespace displayed at the top of the page; this displays the list of available namespaces.

  At the top of the page, click the `Wizards` drop-down list, and select **Data Migration**.
- On the first page of the wizard, select the table or view, as follows:
  - `Select a destination namespace` — Select the InterSystems IRIS namespace to which the data will be copied.
  - `Schema Filter` — Specify a schema (class package) name that contains the table or view. You can specify a name with wildcards to return multiple schemas, or % to return all schemas. For example, C% will return all schemas in the namespace beginning with the letter C. Use of this filter is recommended, as it will shorten the return list of schemas to select from, and thus improve loading speed.
  - `Table Filter` — Specify a table or view name. You can specify a name with wildcards to return multiple tables and/or views, or % to return all tables/views.
  - `Table type` — Select `TABLE`, `VIEW`, `SYSTEM TABLE`, or `ALL`. The default is `TABLE`.
  - `Select a SQL Gateway connection` — Select the SQL Gateway connection to use.
- Click `Next`.
- On the next page, you can optionally specify the following information for each class:
  - `New Schema` — Specify the package to contain the class or classes. Be sure to follow the rules for ObjectScript identifiers, including length limits (see the section on Naming Conventions in *Defining and Using Classes*).

    Tip: To change the package name for all classes, type a value at the top of this column and then click `Change all`.
  - `Copy Definition` — Select this check box to generate this class, based on the table definition in the external source. If you have already generated the class, you can clear this check box.

- `Copy Data` — Select this check box to copy the data for this class from the external source. When you copy data, the wizard overwrites any existing data in the InterSystems IRIS class.

- Click `Next`. The wizard displays the following optional settings:
  - `Disable validation` — If checked, data will be imported with `%NOCHECK` specified in the *restriction* parameter of the **INSERT** command.
  - `Disable journaling for the importing process` — If checked, journaling will be disabled for the process performing the data migration (not system-wide). This can make the migration faster, at the cost of potentially leaving the migrated data in an indeterminate state if the migration is interrupted by a system failure. Journaling is re-enabled at the end of the run, successful or not.
  - `Defer indices` — If checked, indices are built after the data is inserted. The wizard calls the class' **%SortBegin()** method prior to inserting the data in the table. This causes the index entries to be written to a temporary location for sorting. They are written to the actual index location when the wizard calls the **%SortEnd()** method after all rows have been inserted. Do not use Defer Indices if there are Unique indices defined in the table and you want the migration to catch any unique constraint violations. A unique constraint violation will not be caught if Defer Indices is used.
  - `Disable triggers` — If checked, data will be imported with `%NOTRIGGER` specified in the *restriction* parameter of the **INSERT** command.
  - `Delete existing data from table before importing` — If checked, existing data will be deleted rather than merged with the new data.

- Click `Finish`. The wizard opens a new window and displays the Background Jobs page with a link to the background tasks page. Click `Close` to start the import immediately, or click the given link to view the background tasks page. In either case, the wizard starts the import as a background task.
- In the Data Migration Wizard window, click `Done` to go back to the home page of the Management Portal.

Note: The %SQL.Migration.Import class contains wrappers around the Data Migration Wizard. See the class library documentation for details.

4.1 Microsoft Access and Foreign Key Constraints

When you use the Data Migration Wizard with Microsoft Access, the wizard tries to copy any foreign key constraints defined on the Access tables. To do this, it queries the `MSysRelationships` table in Access. By default, this table is hidden and does not provide read access. If the wizard can't access `MSysRelationships`, it migrates the data table definitions to InterSystems SQL without any foreign key constraints.

If you want the utility to migrate the foreign key constraints along with the table definitions, set Microsoft Access to provide read access for `MSysRelationships`, as follows:

- In Microsoft Access, make sure that system objects are displayed.
- Click `Tools > Options` and select the setting on the `View` tab.
- Click `Tools > Security > User and Group Permissions`. Then select the `Read` check box next to the table name.

5

Connecting the SQL Gateway via JDBC

This chapter describes how to create a JDBC logical connection definition for the SQL Gateway. Before following any of the procedures in this chapter, you must first install the Java Runtime Environment on your system. See *Using Java with InterSystems Software* for more information on the InterSystems JDBC driver.

InterSystems IRIS maintains a list of SQL Gateway connection definitions, which are logical names for connections to external data sources. Each connection definition consists of a logical name (for use within InterSystems IRIS), information on connecting to the data source, and a username and password to use when establishing the connection. These connections are stored in the table %Library.sys_SQLConnection. You can export data from this table and import it into another instance of the same version of InterSystems IRIS.

Note: **Controlling SQL Gateway Logging and Other JDBC Settings**

To monitor problems when connected via JDBC, you can enable JDBC logging for the SQL Gateway connection (see "SQL Gateway Logging"). The same dialog also allows you to specify JAVAHOME and other JDBC settings.

In order to establish a SQL Gateway Connection using JDBC, you must have set up the `JAVAHOME` environmental variable so that it points to the Java executable on your system. This can be done via the `Java Home Directory` setting of the %JDBC_Server External Language Server definition, which can be found via System Administration -> Configuration -> Connectivity -> External Language Servers -> %JDBC_Server.

Use of the %JDBC Server requires the associated resource (default resource is %Gateway_SQL).

5.1 Defining a Logical Connection in the Management Portal

To define a SQL Gateway connection for a JDBC-compliant data source, perform the following steps:

1. In the Management Portal, go to the System Administration > Configuration > Connectivity > SQL Gateway Connections page.
2. Click **Create New Connection**.
3. On the **SQL Gateway Connection** page, enter or choose values for the following fields:
   - For **Type**, choose **JDBC**.
   - **Connection Name** — Specify an identifier for the connection, for use within InterSystems IRIS.
   - **User** — Specify the name for the account to serve as the default for establishing connections, if needed.

- **Password** — Specify the password associated with the default account.
- **Driver name** — Full class name of the JDBC client driver.
- **URL** — Connection URL for the data source, in the format required by the JDBC client driver that you are using.
- **Class path** — Specifies a comma-separated list of additional JAR files to load.
- **Properties** — Optional string that specifies vendor-specific connection properties. If specified, this string should be of the following form:

  property= *value*; *property*= *value*;...

  See the JDBC Quick Reference for more information on connection properties.

For example, a typical connection might use the following values:

Setting	Value
Type	`JDBC`
Connection Name	`ConnectionJDBC1`
User	`JDBCUser`
Password	`JDBCPassword`
Driver name	`oracle.jdbc.driver.OracleDriver`
URL	`jdbc:oracle:thin:@//oraserver:1521/SID`
Class path	`/fill/path/to/ojdbc14.jar`
Properties	`oracle.jdbc.V8Compatibility=true; includeSynonyms=false;restrictGetTables=true`

For the other options, see "Implementation-specific JDBC Connection Options."

4. Optionally test if the values are valid. To do so, click the **Test Connection** button. The screen will display a message indicating whether the values you have entered allow for a valid connection.
5. To create the named connection, click **Save**.
6. Click **Close**.

5.2 Creating a Connection between Namespaces

InterSystems IRIS provides JDBC drivers and can be used as a JDBC data source. That is, an InterSystems IRIS instance can connect to itself or to another InterSystems IRIS instance via JDBC and the SQL Gateway. Specifically, the connection is from a namespace in one InterSystems IRIS to a namespace in the other InterSystems IRIS. To connect in this way, you need the same information that you need for any other external database: the connection details for the database driver that you want to use. This section provides the basic information.

5.2.1 Using the SQL Gateway as a JDBC Data Source

To configure one InterSystems IRIS instance (IrisDB-1) to use another separate instance (IrisDB-2) as a JDBC data source, do the following:

1. Within IrisDB-1, use the SQL Gateway to create a JDBC connection to the namespace in IrisDB-2 that you want to use.

   - For **Type**, choose **JDBC**.
   - **Connection Name** — Specify an identifier for the connection, for use within IrisDB-1.
   - **User** — Specify the username needed to access IrisDB-2, if needed.
   - **Password** — Specify the password for this user.
   - **Driver name** — Use com.intersystems.jdbc.IRISDriver
   - **URL** — Connection URL for the data source, in the following format:

     ```
     jdbc:IRIS://IP_address:port/namespace
     ```

     Here *IP_address:port* is the IP address and TCP port where IrisDB-2 is running, and *namespace* is the namespace to which you want to connect (see "Defining a JDBC Connection URL").

   For example, a typical connection might use the following values:

Setting	Value
Type	`JDBC`
Connection Name	`ConnectUser`
User	`_SYSTEM`
Password	`SYS`
Driver name	`com.intersystems.jdbc.IRISDriver`
URL	`jdbc:IRIS://127.0.0.1:1972/User`

   - Class path — Leave this blank.
   - Properties — Optional string that specifies connection properties supported by the InterSystems JDBC drivers. If specified, this string should be of the following form:

     property= *value*`;` *property*= *value*;...

2. For the other options, see "Implementation-specific JDBC Connection Options" later in this section.
3. Click **Save**.
4. Click **Close**.

5.3 Implementation-specific JDBC Connection Options

Before you define an SQL Gateway connection, you should make sure that you understand the requirements of the external database and of the database driver, because these requirements affect how you define the connection.

Do Not Use Delimited Identifiers by Default

The **Do not use delimited identifiers by default** option controls the format of identifiers in the generated routines.

Select this check box if you are using a database that does not support delimited SQL identifiers. This currently includes the following databases:

- Sybase
- Informix

Clear the check box if you are using any other database. All SQL identifiers will be delimited.

Use COALESCE

The **Use COALESCE** option controls how a query is handled when it includes a parameter (`?`), and it has an effect only when a query parameter equals null.

- If you do not select **Use COALESCE** and if a query parameter equals null, the query returns only records that have null for the corresponding value. For example, consider a query of the following form:

  ```
  SELECT ID, Name from LinkedTables.Table WHERE Name %STARTSWITH ?
  ```

  If the provided parameter is null, the query would return only rows with null-valued names.

- If you select **Use COALESCE**, the query wraps each parameter within a `COALESCE` function call, which controls how null values are handled.

  Then, if a query parameter equals null, the query essentially treats the parameter as a wildcard. In the previous example, if the provided parameter is null, this query returns all rows.

Whether you select this option depends on your preferences and on whether the external database supports the `COALESCE` function.

To find out whether the external database supports the `COALESCE` function, consult the documentation for that database.

Conversion in Composite Row IDs

The **Conversion in composite Row IDs** option controls how non-character values are treated when forming a composite ID. Choose an option that is supported by your database:

- **Do not convert non-character values** — This option performs no conversion. This option is suitable only if your database supports concatenating non-character values to character values.
- **Use CAST** — This option uses`CAST` to convert non-character values to character values.
- **Use {fn convert ...}** — This option uses`{fn convert ...}` to convert non-character values to character values.

In all cases, the IDs are concatenated with`||` between the IDs (or transformed IDs).

Consult the documentation for the external database to find out which option or options it supports.

5.4 SQL Gateway Logging

A log can be generated for the SQL Gateway when it is using a JDBC connection. To enable this logging:

- In the Management Portal, go to System Administration > Configuration > External Language Servers.
- Select the **%JDBC_Server** link to bring up the Edit dialog (*caution:* do not select the **%Java_Server** link, which is completely different).

- At the bottom of the Edit dialog, select the `Advanced Settings` **Show** link if advanced settings are not already displayed.
- Specify a name for **LogFile** (for example, jdbcSqlGateway.log) to record the interaction between the SQL Gateway and the database. If you do not specify a fully qualified path, the log file will be in the current directory from when the JDBC SQL Gateway was initially started (probably the /mgr or /mgr/namespace directory.
- Select the Save button at the top of the dialog.
- On the External Server main page, shut down and restart the *%JDBC_Server* connection to enable the new settings.

You can also specify the Java version (JAVAHOME) to be used with the SQL Gateway by setting the **Java Home Directory** field in the edit dialog. See %JDBC Server in the *Configuration Parameter File Reference* for more information about these settings.

Note: Enable logging only when you need to perform troubleshooting. You should not enable logging during normal operation, because it will dramatically slow down performance.

6
Connecting the SQL Gateway via ODBC

InterSystems IRIS maintains a list of SQL Gateway connection definitions, which are logical names for connections to external data sources. Each connection definition consists of a logical name (for use within InterSystems IRIS), information on connecting to the data source, and a username and password to use when establishing the connection. These connections are stored in the table %Library.sys_SQLConnection. You can export data from this table and import it into another instance of the same version of InterSystems IRIS.

6.1 Defining a Logical Connection in the Management Portal

To define a connection for an ODBC-compliant data source, perform the following steps:

1. Define an ODBC data source name (DSN) for the external database (the procedure is probably described in the documentation for that database). .
2. In the Management Portal, go to the System Administration > Configuration > Connectivity > SQL Gateway Connections page.
3. Click **Create New Connection**.
4. On the **Gateway Connection** page, enter or choose values for the following fields:
   - For **Type of connection**, choose **ODBC**.
   - **Connection Name** — Specify an identifier for the connection, for use within InterSystems IRIS.
   - **Select an existing DSN** — Choose the DSN that you previously created. You must use a DSN, since the ODBC SQL Gateway does not support connections without a DSN.
   - **User** — Specify the name for the account to serve as the default for establishing connections, if needed.
   - **Password** — Specify the password associated with the default account.

   For example, a typical connection might use the following values:

Setting	Value
Type	`ODBC`
Connection Name	`ConnectionODBC1`
Select an existing DSN	`MyAccessPlayground`
User	`DBOwner`
Password	`DBPassword`

5. For the other options, see "Implementation-specific ODBC Connection Options" later in this section.
6. Optionally test if the values are valid. To do so, click the **Test Connection** button. The screen will display a message indicating whether the values you have entered in the previous step allow for a valid connection.
7. To create the named connection, click **Save**.
8. Click **Close**.

Note: For OS-specific instructions on how to create a DSN, see the following sections in *Using the InterSystems ODBC Driver*:

- "Defining an ODBC Data Source on Windows"
- "Defining an ODBC Data Source on UNIX®"

6.1.1 Implementation-specific ODBC Connection Options

Before you define an SQL Gateway connection, you should make sure that you understand the requirements of the external database and of the database driver, because these requirements affect how you define the connection. The following options do not apply to all driver implementations.

Legacy Outer Join

The **Enable legacy outer join syntax (Sybase)** option controls whether the connection will enable you use to use legacy outer joins. Legacy outer joins use SQL syntax that predates the SQL-92 standard. To find out whether the external database supports such joins, consult the documentation for that database.

Needs Long Data Length

The **Needs long data length** option controls how the connection will bind data. The value of this option should agree with the `SQL_NEED_LONG_DATA_LEN` setting of the database driver. To find the value of this setting, use the ODBC **SQLGetInfo** function. If `SQL_NEED_LONG_DATA_LEN` equals Y, then select the **Needs long data length** option; otherwise clear it.

Supports Unicode Streams

The **Supports Unicode streams** option controls whether the connection supports Unicode data in streams, which are fields of type LONGVARCHAR or LONGVARBINARY.

- Clear this check box for Sybase. If you are using a Sybase database, all fields you access via the SQL Gateway should include only UTF-8 data.
- Select this check box for other databases.

Do Not Use Delimited Identifiers by Default

The **Do not use delimited identifiers by default** option controls the format of identifiers in the generated routines.

Select this check box if you are using a database that does not support delimited SQL identifiers. This currently includes the following databases:

- Sybase
- Informix

Clear the check box if you are using any other database. All SQL identifiers will be delimited.

Use COALESCE

The **Use COALESCE** option controls how a query is handled when it includes a parameter (`?`), and it has an effect only when a query parameter equals null.

- If you do not select **Use COALESCE** and if a query parameter equals null, the query returns only records that have null for the corresponding value. For example, consider a query of the following form:

  ```
  SELECT ID, Name from LinkedTables.Table WHERE Name %STARTSWITH ?
  ```

  If the provided parameter is null, the query would return only rows with null-valued names.

- If you select **Use COALESCE**, the query wraps each parameter within a `COALESCE` function call, which controls how null values are handled.

  Then, if a query parameter equals null, the query essentially treats the parameter as a wildcard. In the previous example, if the provided parameter is null, this query returns all rows, which is consistent with the behavior of typical ODBC clients.

Whether you select this option depends on your preferences and on whether the external database supports the `COALESCE` function.

To find out whether the external database supports the `COALESCE` function, consult the documentation for that database.

Conversion in Composite Row IDs

The **Conversion in composite Row IDs** option controls how non-character values are treated when forming a composite ID. Choose an option that is supported by your database:

- **Do not convert non-character values** — This option performs no conversion. This option is suitable only if your database supports concatenating non-character values to character values.
- **Use CAST** — This option uses `CAST` to convert non-character values to character values.
- **Use {fn convert ...}** — This option uses `{fn convert ...}` to convert non-character values to character values.

In all cases, the IDs are concatenated with `||` between the IDs (or transformed IDs).

Consult the documentation for the external database to find out which option or options it supports.

6.1.2 Using the SQL Gateway as an ODBC Data Source

InterSystems IRIS provides ODBC drivers and thus can be used as an ODBC data source. That is, an InterSystems IRIS instance can connect to itself or to another InterSystems IRIS instance via ODBC and the SQL Gateway. Specifically, the connection is from a namespace in one InterSystems IRIS to a namespace in the other InterSystems IRIS. To connect in this way, you need the same information that you need for any other external database: the connection details for the database driver that you want to use. This section provides the basic information.

To configure an InterSystems IRIS instance (*InterSystems IRIS_A*) to use another InterSystems IRIS instance (*InterSystems IRIS_B*) as an ODBC data source, do the following:

1. On the machine that is running *InterSystems IRIS_A*, create a DSN that represents the namespace in *InterSystems IRIS_B* that you want to use.

   Tip: If *InterSystems IRIS_B* is installed on this machine, a suitable DSN might already be available, because when you install InterSystems IRIS, the installer automatically creates DSNs.

2. Within *InterSystems IRIS_A*, use the SQL Gateway to create an ODBC connection that uses that DSN. Provide the following details:

   - For **Type**, choose **ODBC**.
   - **Connection Name** — Specify an identifier for the connection, for use within *InterSystems IRIS_A*.
   - **Select an existing DSN** — Choose the DSN that you previously created for *InterSystems IRIS_B*.

   For example, a typical connection might use the following values:

Setting	Value
Type	`ODBC`
Connection Name	`TestConnection`
Select an existing DSN	`TestConnection`

   Tip: You do not need to specify **User** and **Password** because that information is part of the DSN itself.

3. Click **Save**.
4. Click **Close**.